# Project 3: Network Intrusion Detection, Protocol Forensics & Dual-SIEM Threat Hunting

> **Executive Report:** 📄 Download Full Technical Project Report (PDF) *(Link PDF here)*

## 1. Executive Summary & Architecture

As adversaries are increasingly leverage fileless malware and living-off-the-land (LOTL) techniques to evade endpoint security, robust network traffic analysis (NTA) will become a critical fail-safe. The objective of this project was to architect a dedicated Network Intrusion Detection System (NIDS), engineer custom signatures for volumetric network anomalies, and correlate live packet captures with endpoint telemetry across a Dual-SIEM environment.

This project demonstrates the ability to intercept raw Layer 2 traffic via promiscuous sniffing, dissect TCP session states at the packet level, and translate unindexed wire data into actionable threat hunting intelligence using Splunk (SPL) and Wazuh.

## 2. Target Role & Skills Matrix

**Target Role:** Cybersecurity Analyst / Detection Engineer

**Demonstrated Technical Skills:**

- **Network Forensics & Protocol Analysis:** Packet dissection, TCP flag manipulation (`SYN` vs `ACK`), connection state tracking, Wireshark `.pcap` analysis.
- **Detection Engineering:** AF_PACKET promiscuous sniffing, Emerging Threats (ET) rulesets, authoring custom threshold-based Suricata signatures.
- **Dual-SIEM Operations:** Wazuh EVE JSON ingestion pipelines, Splunk Enterprise containerization, structured XML parsing (`KV_MODE`).

- **Threat Hunting (SPL Analytics):** Volumetric anomaly detection (Top Talkers), Long-tail outlier analysis (Rare Processes), Authentication tracking (Windows Event 4624/4625).
- **Linux Systems Administration:** Docker container deployment, Logical Volume Management (LVM) disk expansion, daemon troubleshooting.

## 3. Architectural Topology

Plaintext

```
[Kali Linux (Attacker)] --- (192.168.56.103)
      |
      v
[VirtualBox Host-Only Switch]
      |---> (Promiscuous SPAN/Tap) ---> [Ubuntu 24.04 (Sensor & SIEM Node)]
      |                                  ├── Suricata NIDS (AF_PACKET on enp0s8)
      |                                  ├── Wazuh Agent (Tailing eve.json)
      |                                  └── Splunk Enterprise (Docker)
      v
[Windows Server 2019 (DC01)] --- (192.168.56.110)
      ├── Services: DNS (53), Kerberos (88), SMB (445), LDAP (389)
      └── Telemetry: Sysmon (Event 1) & Security (Event 4624/4625)
```

## Phase 1: Adversarial Emulation (MITRE ATT&CK T1046)

To generate malicious network telemetry, a targeted reconnaissance sweep was executed from Kali Linux against the Domain Controller (DC01). To evade standard application-layer logging (e.g., Windows Event Viewer, IIS logs), a **Half-Open TCP SYN Scan** was utilized.

**Execution:**

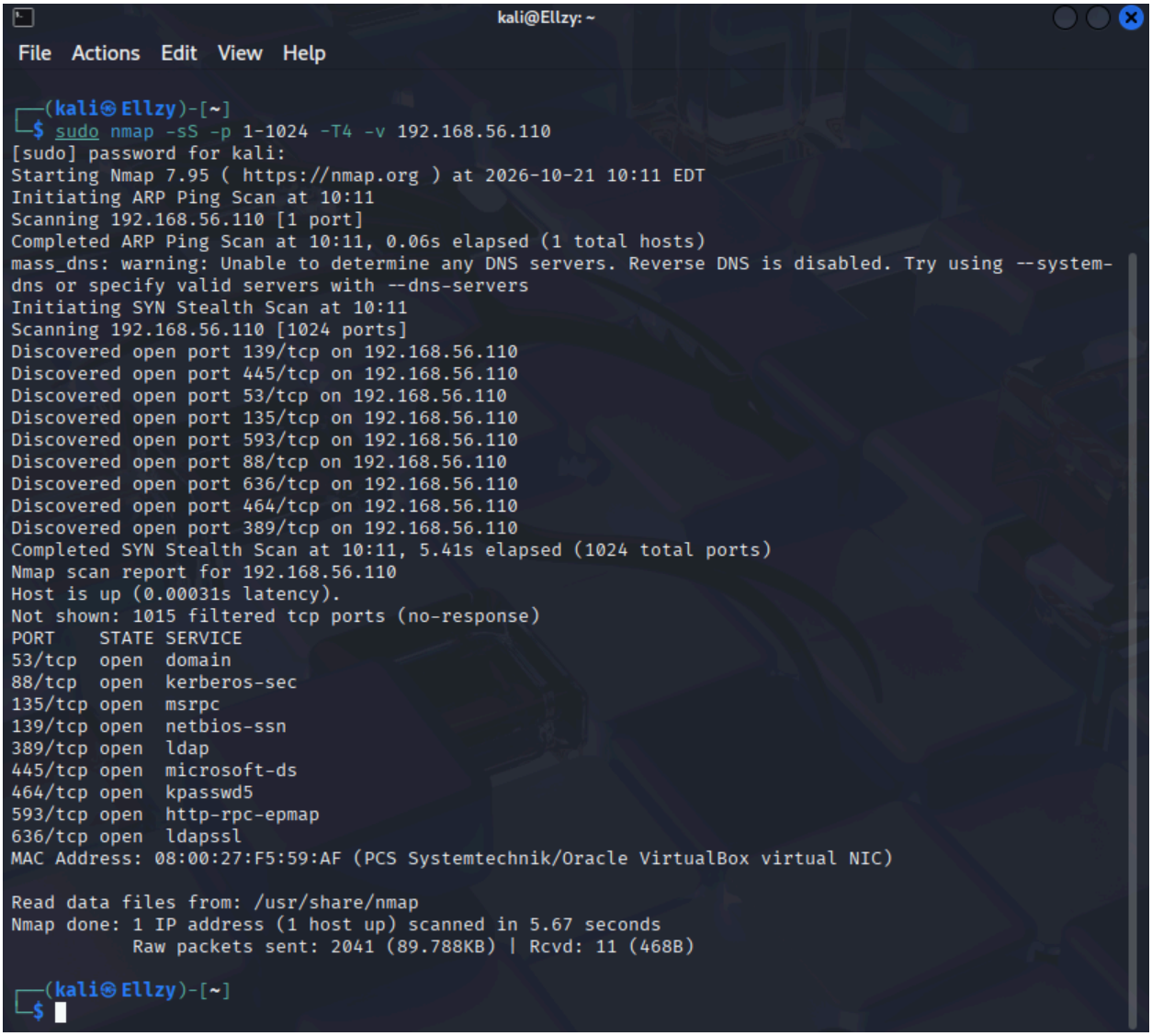
Bash

```
sudo nmap -sS -p 1-1024 -T4 -v 192.168.56.110
```

By manipulating the TCP handshake, the attacker sends only the initial `SYN` packet. When the target replies with `SYN-ACK`, the attacker abruptly sends a `RST` (Reset) packet. Because the session never reaches the `ESTABLISHED` state, the Windows

kernel drops the socket connection before passing it to the application layer, completely bypassing native OS audit logs.

*(Insert Figure 1: Kali terminal executing the Nmap SYN scan)* `![Nmap Scan](IMAGE_LINK_HERE)`




# Phase 2: Protocol Forensics & Packet Dissection

To prove the evasion mechanics at Layer 4, the raw traffic was captured via `tcpdump` on the Ubuntu sensor's promiscuous interface and analyzed in Wireshark.

**Wireshark Display Filter:**

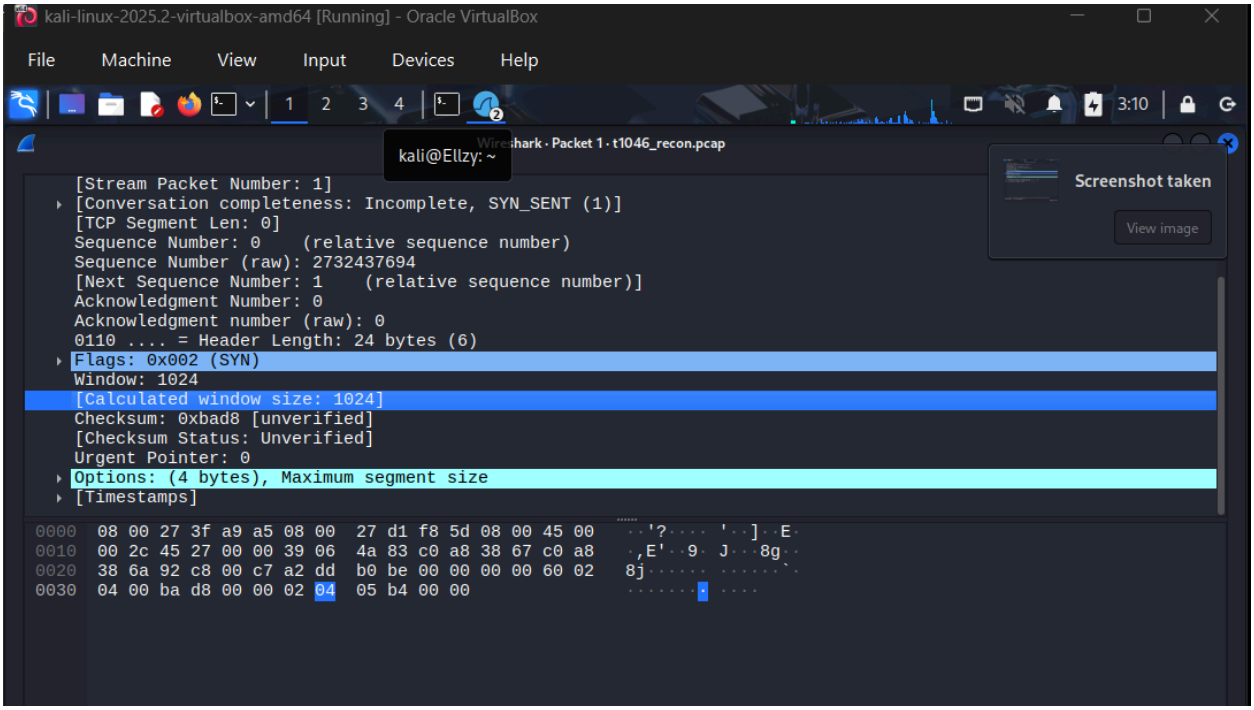
Plaintext

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

**Forensic Findings:**

- **Flag Manipulation:** The packet headers confirmed `Flags: 0x002 (SYN)` with the `Acknowledgment: Not set`.
- **Connection State:** Wireshark's TCP dissector flagged the streams as `[Conversation completeness: Incomplete, SYN_SENT (1)]`.
- **Behavioral Anomaly:** The capture revealed dozens of isolated `SYN` requests originating from a single IP (`192.168.56.103`) targeting sequential ports on `192.168.56.110` within milliseconds—a clear indicator of automated port enumeration.

*(Insert Figure 2: Wireshark GUI displaying the filtered SYN packets and expanded TCP flag details)* `![Wireshark Capture](IMAGE_LINK_HERE)`




# Phase 3: Custom Detection Engineering

While Suricata successfully logged the raw flow data, default signature sets (like Emerging Threats) are typically optimized for payload analysis and exploit

delivery, meaning raw SYN packets often bypass default alert triggers.

To detect this volumetric anomaly, a custom threshold-based signature was authored in `local.rules` to track connection attempt velocity.

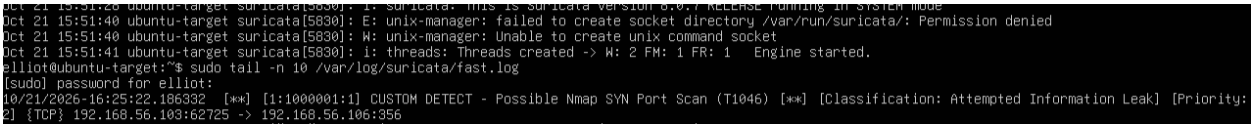
**Suricata Rule (`SID: 1000001`):**

Plaintext

```
alert tcp any any -> $HOME_NET any (msg:"CUSTOM DETECT - Possible Nmap SYN Port Scan (T10
46)"; flags:S,12; threshold:type both, track by_src, count 20, seconds 5; classtype:attem
pted-recon; sid:1000001; rev:1;)
```

**Detection Logic:**

- `flags:S,12`: Inspects the TCP control bits specifically for `SYN`, ignoring sessions containing `ACK` or `RST`.
- `threshold: ... count 20, seconds 5`: Enforces a volumetric state tracker, triggering an alert only when a single source IP (`track by_src`) initiates 20 unique SYN probes within a tight 5-second window, minimizing false positives from normal network jitter.




# Phase 4: Dual-SIEM Correlation & Threat Hunting

With the detection engine validated, telemetry was piped into two distinct SIEM architectures to demonstrate both signature-driven alerting (Wazuh) and statistical threat hunting (Splunk).

## 4.1 Wazuh Pipeline Integration

The Wazuh Agent was configured via `<localfile>` in `ossec.conf` to continuously ingest Suricata's Extensible Event Format (`eve.json`). Wazuh's native decoders automatically parsed the JSON arrays, providing a low-latency, centralized dashboard for immediate triage.




## 4.2 Splunk Analytics & SPL Threat Hunting

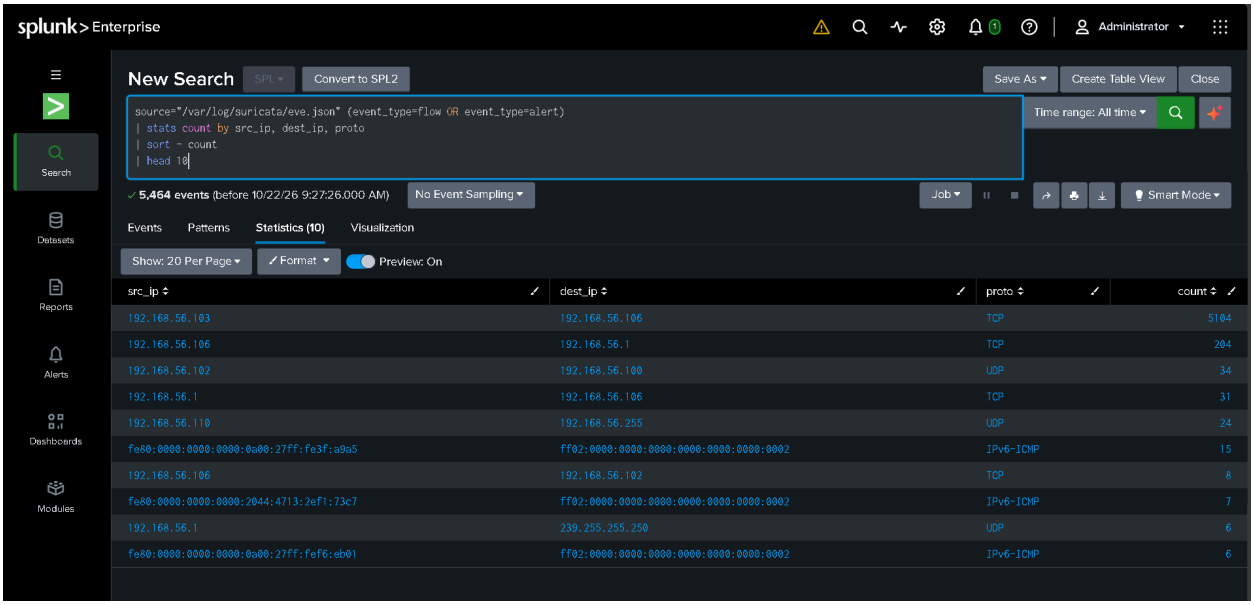
Splunk Enterprise was deployed via Docker directly on the Ubuntu sensor. Telemetry from both the network (Suricata EVE) and the endpoint (Windows Security and Sysmon XML exports) were indexed. Three production-grade hunting queries were executed using Search Processing Language (SPL).

**Hunt 1: Network "Top Talkers" (Volumetric Reconnaissance)**

To isolate the attacker's IP from background network noise, flow telemetry was aggregated to identify hosts generating disproportionate connection attempts.

Splunk SPL

```
source="/var/log/suricata/eve.json" (event_type=flow OR event_type=alert)
| stats count by src_ip, dest_ip, proto
| sort - count
| head 10
```



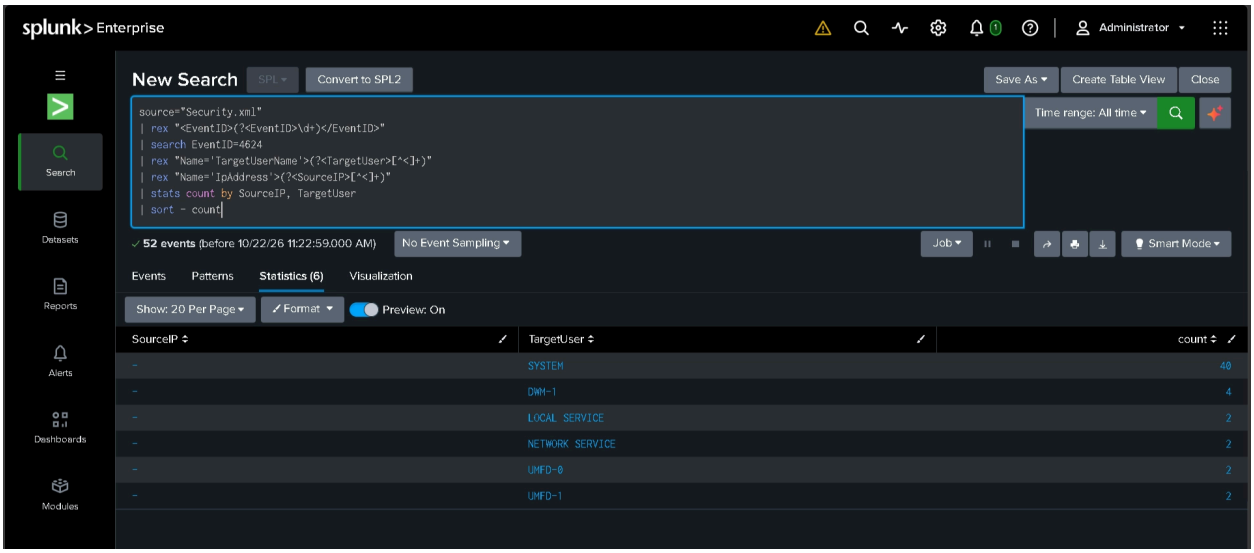

**Hunt 2: Logon Activity Tracking (Credential Abuse)**

Pivoting from network data to endpoint telemetry, Windows Security logs (Event ID 4624/4625) were analyzed to track authentication behavior across the domain, grouping attempts by source IP and target account to identify potential brute-force or password spraying activity.

Splunk SPL

```
source="Security.xml"
| rex "<EventID>(?<EventID>\d+)</EventID>"
| search EventID=4624 OR EventID=4625
| rex "Name='TargetUserName'>(?<TargetUser>[^<]+)"
| rex "Name='IpAddress'>(?<SourceIP>[^<]+)"
| stats count by SourceIP, TargetUser, EventID
| sort - count
```

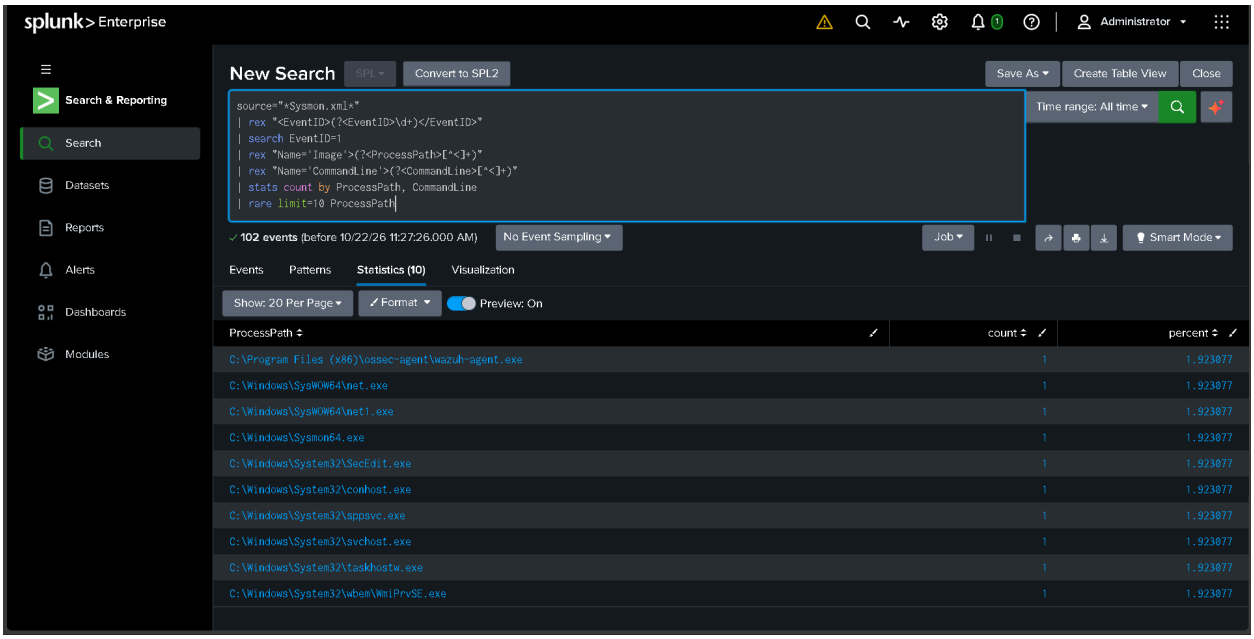
*(Insert Figure 6: Splunk Statistics table displaying authentication counts by TargetUser)* `![Logon Activity](IMAGE_LINK_HERE)`




**Hunt 3: Rare Processes (Long-Tail Anomaly Detection - T1059)**
Because adversaries frequently execute custom payloads or rarely used LOLBins (Living-off-the-Land Binaries), Sysmon Event ID 1 (Process Creation) logs were analyzed using long-tail statistical filtering to isolate outlier executables.

Splunk SPL

```
source="Sysmon.xml"
| rex "<EventID>(?<EventID>\d+)</EventID>"
| search EventID=1
| rex "Name='Image'>(?<ProcessPath>[^<]+)"
| rex "Name='CommandLine'>(?<CommandLine>[^<]+)"
| stats count by ProcessPath, CommandLine
| rare limit=10 ProcessPath
```




# Engineering Challenges & Root Cause Analysis

Building an enterprise-grade environment on limited virtual hardware required resolving several complex system administration roadblocks:

1. **Linux LVM Disk Exhaustion (Splunk Containerization):**
    - **Symptom:** Docker container extraction failed with `no space left on device`.




    - **Root Cause:** The default Ubuntu LVM installation allocated only 12GB to the root volume (`ubuntu--vg-ubuntu--lv`), leaving the remainder of the virtual disk unallocated.



- **Resolution:** Executed `lvextend -l +100%FREE` and `resize2fs` to dynamically expand the live root filesystem to 23GB without downtime, successfully accommodating Splunk's heavy storage requirements.







2. **Splunk Headless License Acceptance:**
   - **Symptom:** Splunk Docker container exited immediately with `License not accepted`.
   - **Root Cause:** Newer Splunk Docker image builds implemented a strict requirement for a secondary license flag covering general terms.



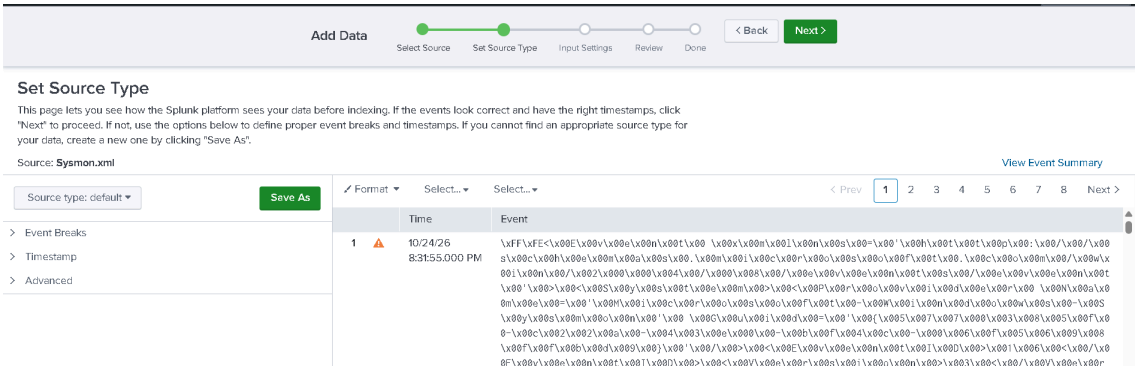

   - **Resolution:** I re-deployed the container with the appended environment variable `e "SPLUNK_GENERAL_TERMS=--accept-sgt-current-at-splunk-com"`.

```
Please read and make sure you agree to the Splunk General Terms before you access or use this software.
Only after doing so should you include the '--accept-sgt-current-at-splunk-com' and '--accept-license' flags to indicate your acceptance of the S
erms and launch this software.
For example: docker run -e SPLUNK_GENERAL_TERMS=--accept-sgt-current-at-splunk-com -e SPLUNK_START_ARGS=--accept-license -e SPLUNK_PASSWORD splur

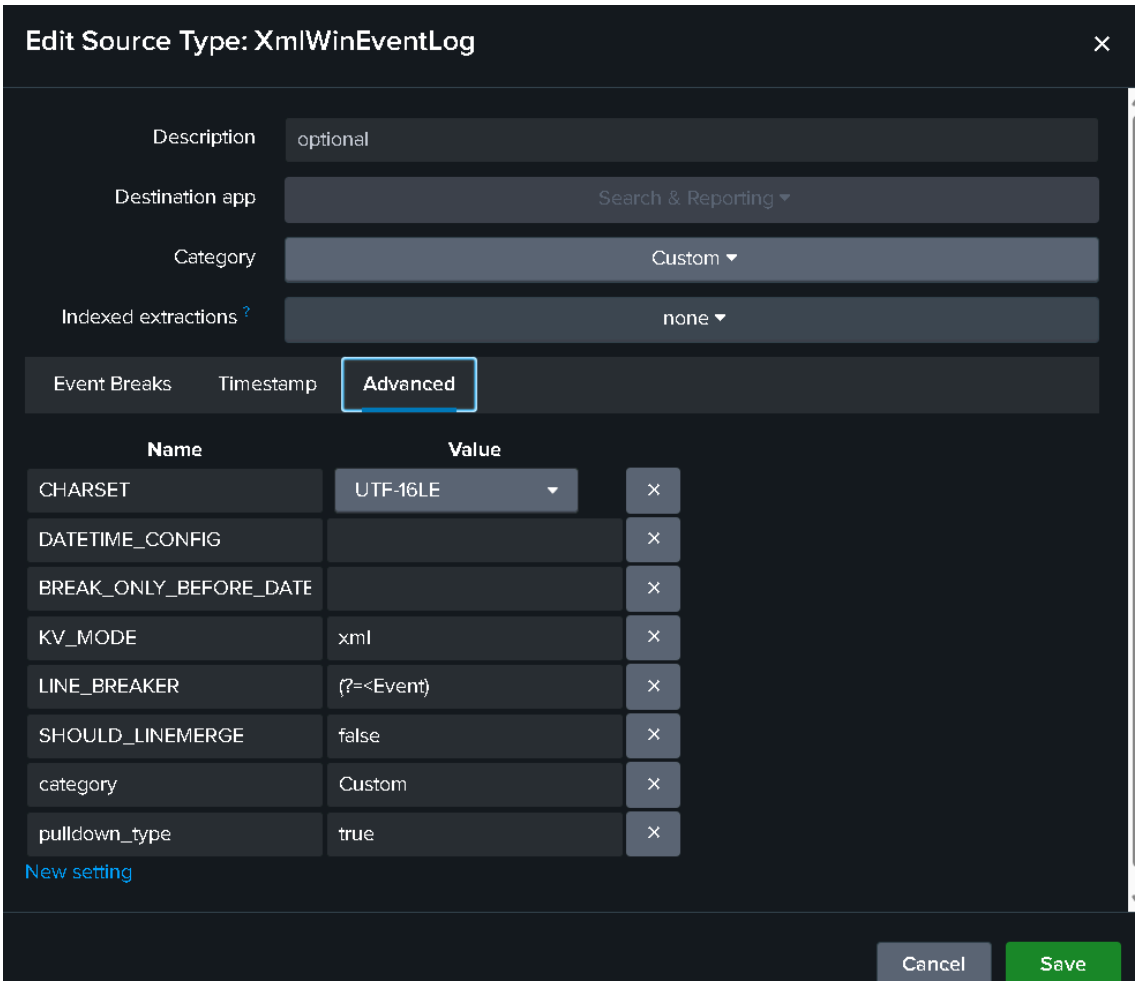
For additional information and examples, see the help: docker run -it splunk/splunk help
elliot@ubuntu-target:~$ sudo docker rm -f splunk 2>/dev/null
splunk
elliot@ubuntu-target:~$ sudo docker run -d \
```

```
elliot@ubuntu-target:~$ sudo docker run -d \
> --name splunk \
> --restart always \
> -p 8000:8000 \
> -e "SPLUNK_GENERAL_TERMS=--accept-sgt-current-at-splunk-com" \
> -e "SPLUNK_START_ARGS=--accept-license" \
> -e "SPLUNK_PASSWORD=EllzySplunk2026!" \
> -v /var/log/suricata:/var/log/suricata:ro \
> splunk/splunk:latest
3840056531b5eb035887153332a1259af31c0b896e3f452aca0a8e0fb0e615a8
elliot@ubuntu-target:~$ _
```

3. **SIEM Binary Decoding Failures (`UTF-16LE`):**
   - **Symptom:** Windows Event Logs (`.evtx`) exported via PowerShell appeared in Splunk as broken binary hex (`\xFF\xFE\x00E...`).




   - **Root Cause:** PowerShell's `Out-File` default encoding utilizes UTF-16LE, which Splunk's Linux backend could not natively parse, treating it as a raw `ElfFile`.
   - **Resolution:** Intercepted the ingestion pipeline at the `props.conf` level via Splunk Web, forcing the `CHARSET` to `UTF-16LE` and configuring `KV_MODE=xml` combined with regex event breaking (`(?=<Event)`) to successfully parse the nested XML schema.


Edit Source Type: XmlWinEventLog
Description
optional
Destination app
Search & Reporting
Category
Custom
Indexed extractions
none
Event Breaks
Timestamp
Advanced
Name
Value
CHARSET
UTF-16LE
DATETIME_CONFIG
BREAK_ONLY_BEFORE_DATE
KV_MODE
xml
LINE_BREAKER
(?=<Event)
SHOULD_LINEMERGE
false
category
Custom
pulldown_type
true
New setting
Cancel
Save